Lab 5: Frameworks and automatic differentiation

Reading and Learning Outcomes

Reading: Reddi, Chapter 7 AI Frameworks: §7.3 Fundamental Concepts, §7.4 Framework Architecture, §7.6 System Integration, and the framework implementation/deployment discussion in §§7.7–7.8. Focus on computation graphs, automatic differentiation, execution modes, tensor representation, and framework overheads.

After completing this lab, students should be able to:

- Implement a scalar automatic-differentiation engine with graph construction.
- Perform topological backpropagation with correct gradient accumulation.
- Validate gradients against PyTorch autograd and numerical estimates.
- Assess eager versus compiled or traced execution and the cost of non-contiguous tensors.

1. [25] Design a `Value` node that stores data, gradient, parent nodes, and local derivative rule.
2. [20] Implement addition, multiplication, powers, and ReLU. Using them must build a computation graph during forward pass.
3. [25] Implement backward pass:
   - Topologically sort the graph.
   - Set output gradient to 1.
   - Visit nodes in reverse order.
   - Accumulate gradients, do not overwrite.
4. [10] Verify against PyTorch autograd and numerical gradients on `((a*b + c)**2).relu().sum()`. Max absolute difference must be `<1e-6`.
5. [15] Train a tiny model on `make_moons`. Architecture: 2 → 8 → 1, sigmoid, BCE loss. Compare speed with PyTorch. Median of 5 runs.
6. [20] In PyTorch, inspect the graph of a small model. Compare eager execution against `torch.compile` or `torch.jit.trace` after warm-up. If compilation is unsupported, use tracing or scripting. Inspect tensor shape, dtype, contiguity. Measure how a non-contiguous view changes operation speed.

Graded evidence: autograd engine matching PyTorch, eager vs compiled timings, short note on when compilation helps.